Mini-SIEM – Vollständiger Projektplan

Überblick

Attribut	Wert
Projektname	Mini-SIEM
Sprache	Python 3.12+
Ziel	Logs sammeln, korrelieren, Angriffe erkennen & visualisieren
Gesamtdauer	~4 Wochen (Teilzeit)
Schwierigkeitsgrad	Mittel

Phase 1 – Fundament (Woche 1)

Ziel

Lauffähige Entwicklungsumgebung + erster Parser + erste Tests.

Aufgaben

1.1 Projektstruktur anlegen

```
mini-siem/
├── src/siem/
│   ├── collector/
│   ├── normalizer/
│   ├── engine/rules/
│   ├── alerts/
│   ├── storage/
│   └── dashboard/
├── tests/unit/
├── tests/integration/
├── config/
├── scripts/
├── docker/
└── docs/
```

```
mkdir mini-siem && cd mini-siem
git init
python -m venv .venv && source .venv/bin/activate
mkdir -p src/siem/{collector,normalizer,engine/rules,alerts,storage,dashboard}
mkdir -p tests/{unit,integration} config docs scripts docker
```

1.2 Abhängigkeiten definieren (`pyproject.toml`)

```
[project]
name = "mini-siem"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "watchdog>=4.0",
    "sqlalchemy>=2.0",
    "alembic>=1.13",
    "flask>=3.0",
    "redis>=5.0",
    "pyyaml>=6.0",
    "python-dotenv>=1.0"
]

[project.optional-dependencies]
dev = ["pytest", "pytest-cov", "ruff", "black", "pre-commit"]
```

1.3 Normalizer implementieren

Abstrakte Basisklasse + Apache-Parser als erstes Beispiel.

```
# src/siem/normalizer/base.py
from abc import ABC, abstractmethod

class BaseParser(ABC):
    @abstractmethod
    def parse(self, raw: str) -> dict | None:
        """Gibt normalisiertes Event-Dict zurück oder None."""
        pass
```

```
# src/siem/normalizer/apache_parser.py
import re
from .base import BaseParser

PATTERN = re.compile(
    r'(?P<ip>\S+) \S+ \S+ \[(?P<time>[^\]]+)\] '
    r'"(?P<method>\S+) (?P<path>\S+)[^"]*" '
    r'(?P<status>\d{3}) (?P<size>\S+)'
)

class ApacheParser(BaseParser):
    def parse(self, raw: str) -> dict | None:
        m = PATTERN.match(raw)
        if not m:
```

```
            return None
        return {
            "source_ip":  m.group("ip"),
            "timestamp":  m.group("time"),
            "method":     m.group("method"),
            "path":       m.group("path"),
            "status":     int(m.group("status")),
            "event_type": self._classify(int(m.group("status"))),
        }

    def _classify(self, status: int) -> str:
        if status == 401: return "LOGIN_FAILED"
        if status >= 500: return "SERVER_ERROR"
        if status >= 400: return "CLIENT_ERROR"
        return "SUCCESS"
```

1.4 Erste Tests schreiben

```
# tests/unit/test_apache_parser.py
import pytest
from siem.normalizer.apache_parser import ApacheParser

parser = ApacheParser()

def test_parses_valid_line():
    line = '192.168.1.1 - - [01/Mar/2026:10:00:00 +0000] "GET /login HTTP/1.1" 401
512'
    result = parser.parse(line)
    assert result["source_ip"] == "192.168.1.1"
    assert result["event_type"] == "LOGIN_FAILED"

def test_returns_none_for_garbage():
    assert parser.parse("das ist kein log") is None
```

Deliverables Phase 1

- [ ] Git-Repo mit vollständiger Ordnerstruktur
- [ ] `ApacheParser` mit Tests (grüne CI)
- [ ] `NginxParser` und `SyslogParser` (analog)
- [ ] `.env.example` und `config/settings.yml`

Phase 2 – Collector + Storage (Woche 2)

Ziel

Logs aus Dateien einlesen und in SQLite speichern.

2.1 File Watcher (Collector)

```python
# src/siem/collector/file_watcher.py
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
from siem.normalizer.apache_parser import ApacheParser

class LogFileHandler(FileSystemEventHandler):
    def __init__(self, parser, on_event):
        self.parser = parser
        self.on_event = on_event
        self._offsets = {}

    def on_modified(self, event):
        if not event.src_path.endswith(".log"):
            return
        path = event.src_path
        offset = self._offsets.get(path, 0)
        with open(path, "r") as f:
            f.seek(offset)
            for line in f:
                event_dict = self.parser.parse(line.strip())
                if event_dict:
                    self.on_event(event_dict)
            self._offsets[path] = f.tell()

def watch(path: str, parser, on_event):
    handler = LogFileHandler(parser, on_event)
    observer = Observer()
    observer.schedule(handler, path=path, recursive=False)
    observer.start()
    return observer
```

2.2 Datenbank-Modelle (SQLAlchemy)

```python
# src/siem/storage/models.py
from sqlalchemy import Column, Integer, String, DateTime, Enum
from sqlalchemy.orm import DeclarativeBase
from datetime import datetime

class Base(DeclarativeBase):
    pass

class Event(Base):
    __tablename__ = "events"
    id         = Column(Integer, primary_key=True)
    source_ip  = Column(String(45))
    event_type = Column(String(50))
    raw_log    = Column(String(2000))
    created_at = Column(DateTime, default=datetime.utcnow)

class Alert(Base):
```

```
    __tablename__ = "alerts"
    id         = Column(Integer, primary_key=True)
    alert_type = Column(String(50))
    source_ip  = Column(String(45))
    severity   = Column(Enum("LOW", "MEDIUM", "HIGH", "CRITICAL"))
    message    = Column(String(500))
    created_at = Column(DateTime, default=datetime.utcnow)
```

```
# src/siem/storage/repository.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from .models import Base, Event, Alert

class Repository:
    def __init__(self, db_url: str = "sqlite:///siem.db"):
        engine = create_engine(db_url)
        Base.metadata.create_all(engine)
        self.Session = sessionmaker(bind=engine)

    def save_event(self, data: dict):
        with self.Session() as s:
            s.add(Event(**data))
            s.commit()

    def save_alert(self, data: dict):
        with self.Session() as s:
            s.add(Alert(**data))
            s.commit()

    def get_recent_alerts(self, limit: int = 100) -> list:
        with self.Session() as s:
            return
s.query(Alert).order_by(Alert.created_at.desc()).limit(limit).all()
```

2.3 Pipeline zusammenbauen

```
# src/siem/pipeline.py
from siem.collector.file_watcher import watch
from siem.normalizer.apache_parser import ApacheParser
from siem.engine.rule_engine import RuleEngine
from siem.storage.repository import Repository

def run(log_path: str):
    repo   = RuleEngine()
    engine = RuleEngine()

    def handle_event(event: dict):
        repo.save_event(event)
        alert = engine.evaluate(event)
```

```
        if alert:
            repo.save_alert(alert)

    observer = watch(log_path, ApacheParser(), handle_event)
    try:
        while observer.is_alive():
            observer.join(timeout=1)
    except KeyboardInterrupt:
        observer.stop()
```

Deliverables Phase 2

- [ ] `FileWatcher` liest Logs in Echtzeit
- [ ] SQLite-Datenbank mit `events` und `alerts` Tabellen
- [ ] End-to-End-Test: Log-Zeile → DB-Eintrag
- [ ] `scripts/generate_test_logs.py` für Testdaten

Phase 3 – Correlation Engine & Regeln (Woche 3, Teil 1)

Ziel

Angriffsmuster automatisch erkennen.

3.1 Rule Engine

```
# src/siem/engine/rule_engine.py
import importlib
import pkgutil
from siem.engine import rules as rules_pkg

class RuleEngine:
    def __init__(self):
        self.rules = self._load_rules()

    def _load_rules(self):
        loaded = []
        for _, name, _ in pkgutil.iter_modules(rules_pkg.__path__):
            module = importlib.import_module(f"siem.engine.rules.{name}")
            if hasattr(module, "check"):
                loaded.append(module.check)
        return loaded

    def evaluate(self, event: dict) -> dict | None:
        for rule in self.rules:
            result = rule(event)
            if result:
                return result
        return None
```

3.2 Erkennungsregeln

Brute-Force-Erkennung

```
# src/siem/engine/rules/brute_force.py
from collections import defaultdict
from datetime import datetime, timedelta

_failed = defaultdict(list)
THRESHOLD = 5
WINDOW = 60  # Sekunden

def check(event: dict) -> dict | None:
    if event.get("event_type") != "LOGIN_FAILED":
        return None

    ip  = event["source_ip"]
    now = datetime.utcnow()
    cutoff = now - timedelta(seconds=WINDOW)

    _failed[ip] = [t for t in _failed[ip] if t > cutoff]
    _failed[ip].append(now)

    if len(_failed[ip]) >= THRESHOLD:
        return {
            "alert_type": "BRUTE_FORCE",
            "source_ip":  ip,
            "severity":   "HIGH",
            "message":    f"{len(_failed[ip])} fehlgeschlagene Logins in
{WINDOW}s"
        }
```

Port-Scan-Erkennung

```
# src/siem/engine/rules/port_scan.py
from collections import defaultdict
from datetime import datetime, timedelta

_connections = defaultdict(set)
THRESHOLD = 20
WINDOW = 10

def check(event: dict) -> dict | None:
    if "port" not in event:
        return None

    ip  = event["source_ip"]
    now = datetime.utcnow()
    _connections[ip].add((event["port"], now))
```

```
    recent_ports = {p for p, t in _connections[ip]
                    if t > now - timedelta(seconds=WINDOW)}

    if len(recent_ports) >= THRESHOLD:
        return {
            "alert_type": "PORT_SCAN",
            "source_ip":  ip,
            "severity":   "MEDIUM",
            "message":    f"{len(recent_ports)} Ports in {WINDOW}s gescannt"
        }
```

SQL-Injection-Erkennung

```
# src/siem/engine/rules/sql_injection.py
import re

PATTERN = re.compile(
    r"(union\s+select|drop\s+table|insert\s+into|--|;--|/\*|\bor\b\s+1=1)",
    re.IGNORECASE
)

def check(event: dict) -> dict | None:
    path = event.get("path", "")
    if PATTERN.search(path):
        return {
            "alert_type": "SQL_INJECTION",
            "source_ip":  event.get("source_ip", "unknown"),
            "severity":   "CRITICAL",
            "message":    f"Verdächtiger Pfad: {path[:200]}"
        }
```

3.3 Regeln per YAML konfigurieren

```
# config/rules.yml
rules:
  brute_force:
    enabled: true
    threshold: 5
    window_seconds: 60
    severity: HIGH

  port_scan:
    enabled: true
    threshold: 20
    window_seconds: 10
    severity: MEDIUM

  sql_injection:
```

```
    enabled: true
    severity: CRITICAL
```

Deliverables Phase 3a

- [ ] `RuleEngine` mit Plugin-System (auto-lädt Regeln)
- [ ] 3 Erkennungsregeln: Brute-Force, Port-Scan, SQL-Injection
- [ ] Regeln per `rules.yml` aktivierbar/deaktivierbar
- [ ] Unit-Tests für jede Regel

Phase 4 – Dashboard & Alerts (Woche 3, Teil 2)

Ziel

Alerts sichtbar machen und Benachrichtigungen versenden.

4.1 Flask-Dashboard

```python
# src/siem/dashboard/routes.py
from flask import Flask, render_template, jsonify
from siem.storage.repository import Repository

app   = Flask(__name__)
repo  = Repository()

@app.route("/")
def index():
    alerts = repo.get_recent_alerts(limit=50)
    return render_template("dashboard.html", alerts=alerts)

@app.route("/api/alerts")
def api_alerts():
    alerts = repo.get_recent_alerts(limit=100)
    return jsonify([{
        "type":      a.alert_type,
        "ip":        a.source_ip,
        "severity":  a.severity,
        "message":   a.message,
        "timestamp": a.created_at.isoformat()
    } for a in alerts])
```

4.2 Alert-Kanäle

```python
# src/siem/alerts/slack_alert.py
import requests

class SlackAlert:
```

```
    def __init__(self, webhook_url: str):
        self.url = webhook_url

    COLORS = {"LOW": "#36a64f", "MEDIUM": "#ffcc00",
              "HIGH": "#ff6600", "CRITICAL": "#ff0000"}

    def send(self, alert: dict):
        payload = {
            "attachments": [{
                "color": self.COLORS.get(alert["severity"], "#cccccc"),
                "title": f"🚨 {alert['alert_type']}",
                "text":  f"IP: `{alert['source_ip']}`\n{alert['message']}",
            }]
        }
        requests.post(self.url, json=payload, timeout=5)
```

Deliverables Phase 4

- ☐ Flask-Dashboard mit Alert-Tabelle
- ☐ Live-Refresh per AJAX (alle 10 Sek.)
- ☐ Slack-/E-Mail-Benachrichtigung bei CRITICAL
- ☐ Severity-Farben im UI (ROT/ORANGE/GELB/GRÜN)

Phase 5 – Docker & CI/CD (Woche 4)

Ziel

Produktionsreifes Deployment und automatisierte Tests.

5.1 Dockerfile

```
FROM python:3.12-slim

WORKDIR /app
COPY pyproject.toml .
RUN pip install -e .

COPY src/ src/
COPY config/ config/

CMD ["python", "-m", "siem.pipeline"]
```

5.2 Docker Compose

```
# docker/docker-compose.yml
services:
  siem:
```

```yaml
    build: ..
    env_file: ../.env
    volumes:
      - /var/log:/logs:ro
    depends_on: [db, redis]
    ports:
      - "5000:5000"

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: siem
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine

volumes:
  pgdata:
```

5.3 GitHub Actions CI

```yaml
# .github/workflows/ci.yml
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -e ".[dev]"
      - run: ruff check src/
      - run: pytest tests/ --cov=siem --cov-report=xml
      - uses: codecov/codecov-action@v4
```

Deliverables Phase 5

- ☐ Docker-Image baut ohne Fehler
- ☐ `docker compose up` startet alles
- ☐ CI läuft bei jedem Push durch
- ☐ Code-Coverage > 70%

Gesamtzeitplan

Woche	Phase	Hauptaufgaben
1	Fundament	Projektstruktur, Normalizer, erste Tests
2	Collector + Storage	File Watcher, SQLite, Pipeline
3a	Correlation Engine	RuleEngine, 3 Erkennungsregeln
3b	Dashboard + Alerts	Flask-UI, Slack-Notification
4	Docker + CI/CD	Dockerfile, GitHub Actions

Erkennungsregeln – Übersicht

Regel	Trigger	Schwere
Brute-Force	>5 Fehlanmeldungen in 60s	HIGH
Port-Scan	>20 Ports in 10s	MEDIUM
SQL-Injection	Verdächtige URL-Parameter	CRITICAL
Directory Traversal	`../` im Pfad	HIGH
Anomalie-Login	Login außerhalb 08–18 Uhr	LOW

Empfohlene Tools & Libraries

Kategorie	Tool	Zweck
Log-Watching	`watchdog`	Datei-Änderungen erkennen
Datenbank	`sqlalchemy` + `alembic`	ORM + Migrationen
Web-UI	`flask`	Dashboard
Queue	`redis`	Event-Buffer
Testing	`pytest` + `pytest-cov`	Tests + Coverage
Linting	`ruff` + `black`	Code-Qualität
Secrets	`python-dotenv`	.env-Verwaltung

Erweiterungsideen (nach v1.0)

- **Syslog-Receiver** (UDP Port 514) für Netzwerkgeräte
- **GeoIP-Lookup** für Angreifer-Standort
- **Anomalie-Erkennung** mit Machine Learning (Isolation Forest)
- **MITRE ATT&CK Mapping** für erkannte Techniken
- **API-Integration** (z.B. VirusTotal für IP-Reputation)

- **Elasticsearch** statt SQLite für große Log-Mengen

Erstellt: März 2026 | Version 1.0